**Campus Event Management Platform**

CS 202C course project ▪ IIT Dharwad

# Developer Guide

Architecture, code reference and conventions


| | |
|---|---|
| **Author** | Prajyot (MC25B012) |
| **Course** | CS 202C, Indian Institute of Technology Dharwad |
| **Document** | CEMP-DG-01 |
| **Version** | 1.0, November 2026 |
| **Software** | Flask + SQLite web application (Python 3.10+) |



Part of the project documentation set: Project Report • Presentation Slides • User Manual • Installation & Operations Guide • Developer Guide • Test Plan & Report. Source: `docs/` and `report/` in the project repository.

# Contents

**1 Introduction** **2**
1.1 Design principles . . . 2
1.2 Technology stack . . . 2

**2 Repository layout** **2**

**3 Architecture** **3**
3.1 Overview . . . 3
3.2 Life of a request . . . 3
3.3 Application factory . . . 4

**4 Database design** **4**
4.1 Entity–relationship model . . . 4
4.2 Table reference . . . 5
4.3 Conventions . . . 5
4.4 Indexes . . . 6

**5 Route reference** **6**

**6 Module reference** **7**
6.1 `app/db.py`: database access . . . 7
6.2 `app/auth.py`: accounts and access control . . . 7
6.3 `app/events.py`: browsing and registrations . . . 8
6.4 `app/admin.py`: administration . . . 9
6.5 `app/reports.py`: report queries . . . 9
6.6 `app/seed.py`: demo data . . . 10

**7 Business rules** **10**
7.1 Registration . . . 10
7.2 Event validation (`read_event_form`) . . . 11
7.3 The overlap rule . . . 11
7.4 Venue rules . . . 11

**8 Templates and styling** **11**
8.1 Template inheritance . . . 11
8.2 CSS . . . 12

**9 Security model** **12**

**10 Testing** **12**

**11 Profiling toolkit** **12**

**12 Extending the application** **13**
12.1 Add a page . . . 13
12.2 Change the schema . . . 13
12.3 Add a category or a report . . . 13
12.4 Coding conventions . . . 13
12.5 Git workflow . . . 14

# 1 Introduction

This guide is the technical reference for the Campus Event Management Platform. It explains how the code is organised, what every module, route and function does, how the database is designed, and how to extend the application safely. It is written for someone reading or changing the code. How to *use* the site is in the User Manual; how to install it is in the Installation & Operations Guide; the tests are documented in the Test Plan & Report.

## 1.1 Design principles

1. **Simple and explainable.** Plain functions, short modules, comments that say *why*. No clever abstractions; every line should be explainable in a viva.
2. **Raw SQL, no ORM.** All database work uses Python's built-in `sqlite3` module with hand-written SQL, so the queries can be read, tested, profiled and `EXPLAIN`ed.
3. **Every value through a placeholder.** User input reaches SQLite only through `?` parameters, never through string formatting.
4. **The database enforces the rules too.** `UNIQUE`, `CHECK` and foreign-key constraints back up the Python validation.
5. **Hand-written HTML and CSS.** Jinja2 templates and one stylesheet; no CSS or JavaScript framework.
6. **Measure before optimising.** Performance changes are justified by `cProfile` measurements and query plans kept in `profiling/`.

## 1.2 Technology stack

| Layer | Technology |
|---|---|
| Language | Python 3.10+ |
| Web framework | Flask 3 (routing, Blueprints, Jinja2 templates, Click CLI) |
| Authentication | Flask-Login (sessions), Werkzeug `generate_password_hash` / `check_password_hash` |
| Database | SQLite through `sqlite3`, raw SQL |
| Front end | Jinja2 HTML templates, one hand-written CSS file |
| Testing | pytest with Flask's test client |
| Demo data | Faker (`en_IN` locale, fixed seed) |
| Profiling | `cProfile`, `pstats`, SnakeViz, `EXPLAIN QUERY PLAN` |
| Automation | Bash scripts |
| Documentation | LaTeX (article and Beamer), Markdown |

Table 1: Technology stack.

# 2 Repository layout

```
Campus-Event-Management-Platform/
|-- app/                        Flask application package
|   |-- __init__.py             create_app(): config, blueprints, error pages, home page
|   |-- db.py                   get_db/close_db, init-db, add-indexes, create-admin commands
|   |-- auth.py                 User class, login manager, admin_required, register/login/logout
|   |-- events.py               search, event detail, register/cancel, my registrations
|   |-- admin.py                dashboard, venue + event CRUD, clash check, participants, CSV
|   |-- reports.py              report queries (JOIN + GROUP BY)
|   |-- seed.py                 "flask seed" demo-data command
|   |-- templates/              Jinja2 templates: base, index, error, auth/, events/, admin/
|   `-- static/style.css        the only stylesheet
```

```
|-- sql/
|   |-- schema.sql            tables and constraints
|   `-- indexes.sql           performance indexes (added after profiling)
|-- scripts/                  Bash automation: common, setup, run, seed, test, profile,
    build_docs
|-- tests/                    conftest.py (fixtures) + 5 test files, 45 tests
|-- profiling/                run_profile.py, explain.py, RESULTS.md, before/ rewrite_only/ after
    /
|-- report/                   report.tex, slides.tex, figures/
|-- docs/                     campusdoc.sty + the four guides (this file is one of them)
|-- instance/                 campus.db (created by setup.sh, git-ignored)
|-- requirements.txt  pytest.ini  .gitignore  .gitattributes
`-- README.md  AI_USAGE.md  CLAUDE.md
```

Listing 1: Project tree (generated files such as venv/ and instance/ are git-ignored).

# 3 Architecture

## 3.1 Overview

The application follows the usual three-tier structure of a server-rendered web app: **presentation** (Jinja2 templates and CSS), **application logic** (Flask Blueprints and plain Python functions) and **data** (SQLite). There is no client-side JavaScript apart from the browser's built-in `confirm()` on delete buttons.

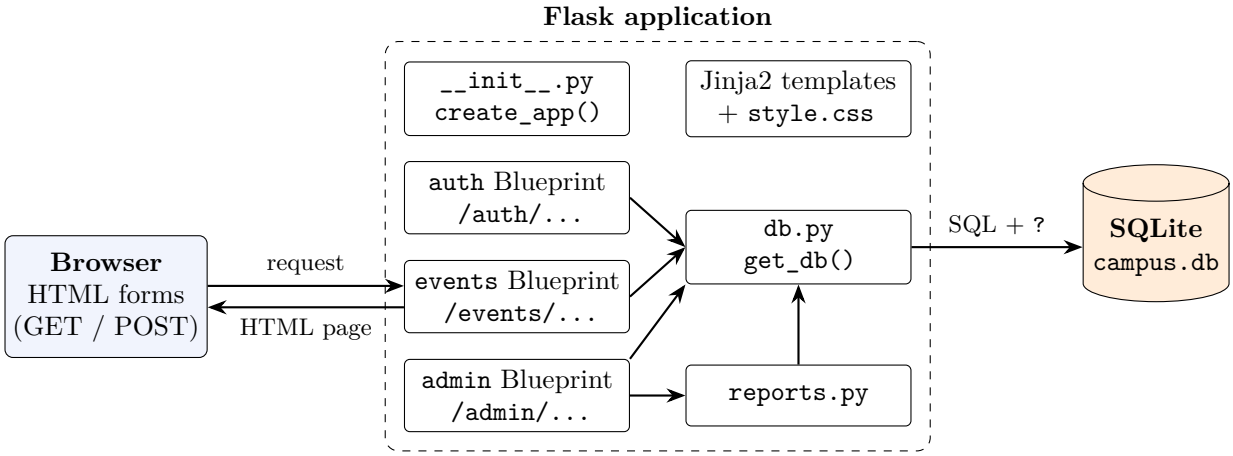



Figure 1: Components and how a request flows through them.

## 3.2 Life of a request

Taking "a student clicks Register" as an example:

1. The browser sends `POST /events/7/register` with the session cookie.
2. Flask matches the URL rule `/events/<int:event_id>/register`, so it calls the view function `register_for_event` in `events.py` with `event_id=7`.
3. `@login_required` asks Flask-Login for the current user. Flask-Login reads the user id from the signed cookie and calls `load_user()`, which runs one `SELECT` on `users`. A visitor is redirected to `/auth/login`.
4. The view calls `get_event(7)`. `get_db()` opens a connection the first time it is needed in this request and stores it in `flask.g`.

5. The view checks the registration rules (Section 7), runs the `INSERT`, commits, and `flash()`es a message.
6. It returns a redirect to the detail page (*Post/Redirect/Get*), so refreshing the page never submits the form twice.
7. When the request ends, `close_db()` (registered with `teardown_appcontext`) closes the connection.
8. The browser follows the redirect; the detail page renders the flashed message from `base.html`.

### 3.3 Application factory

`create_app(test_config=None)` in `app/__init__.py` builds a new Flask application:

1. creates `Flask(__name__, instance_relative_config=True)`, so `app.instance_path` is the `instance/` folder next to `app/`;
2. sets `SECRET_KEY` (env var, default `dev-change-me`), `DATABASE` (env var `CAMPUS_DB`, default `instance/campus.db`) and `SESSION_COOKIE_SAMESITE="Lax"`;
3. applies `test_config` on top (the tests pass a temporary database path);
4. creates the `instance/` folder;
5. registers the database helpers and CLI commands (`db.init_app`) and the `seed` command;
6. initialises Flask-Login and registers the `auth`, `events` and `admin` Blueprints;
7. defines the home page route `/` and the 403 and 404 error handlers, which render `error.html`.

Building the app in a function instead of at import time is what lets every test get its own isolated app and database.

## 4 Database design

### 4.1 Entity–relationship model

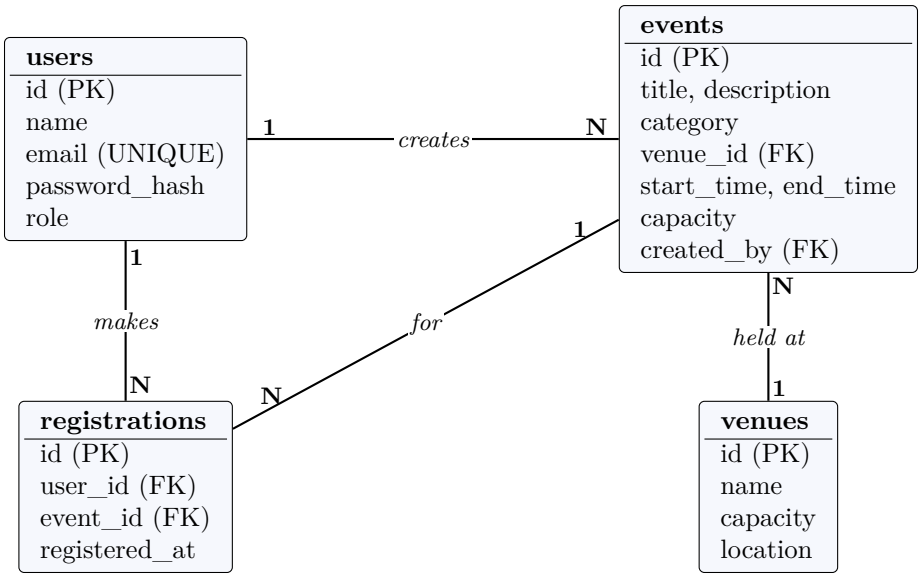



Figure 2: Four tables. `registrations` links students and events (many-to-many).

## 4.2 Table reference

The schema is in `sql/schema.sql`. Running it drops and recreates all tables, children first.

**users** : everyone who can log in.

| Column | Type | Constraints and meaning |
|---|---|---|
| `id` | `INTEGER` | Primary key, auto-increment. |
| `name` | `TEXT` | `NOT NULL`. Display name. |
| `email` | `TEXT` | `NOT NULL UNIQUE`. Stored lower-case by the app; the login name. |
| `password_hash` | `TEXT` | `NOT NULL`. Werkzeug salted hash (scrypt by default); never the password. |
| `role` | `TEXT` | `NOT NULL DEFAULT 'student'`, `CHECK (role IN ('admin','student'))`. |

**venues** : physical places.

| Column | Type | Constraints and meaning |
|---|---|---|
| `id` | `INTEGER` | Primary key, auto-increment. |
| `name` | `TEXT` | `NOT NULL`. |
| `capacity` | `INTEGER` | `NOT NULL`, `CHECK (capacity > 0)`. |
| `location` | `TEXT` | `NOT NULL`. Building and floor. |

**events** : things students register for.

| Column | Type | Constraints and meaning |
|---|---|---|
| `id` | `INTEGER` | Primary key, auto-increment. |
| `title` | `TEXT` | `NOT NULL`. |
| `description` | `TEXT` | Optional. |
| `category` | `TEXT` | `NOT NULL`. One of `CATEGORIES` (checked in Python). |
| `venue_id` | `INTEGER` | `NOT NULL`, foreign key to `venues(id)`. |
| `start_time` | `TEXT` | `NOT NULL`. `'YYYY-MM-DD HH:MM'`. |
| `end_time` | `TEXT` | `NOT NULL`. Same format. Table check `CHECK (end_time > start_time)`. |
| `capacity` | `INTEGER` | `NOT NULL`, `CHECK (capacity > 0)`. At most the venue's capacity (checked in Python). |
| `created_by` | `INTEGER` | `NOT NULL`, foreign key to `users(id)`: the admin who created it. |

**registrations** : which student is registered for which event.

| Column | Type | Constraints and meaning |
|---|---|---|
| `id` | `INTEGER` | Primary key, auto-increment. |
| `user_id` | `INTEGER` | `NOT NULL`, foreign key to `users(id) ON DELETE CASCADE`. |
| `event_id` | `INTEGER` | `NOT NULL`, foreign key to `events(id) ON DELETE CASCADE`. |
| `registered_at` | `TEXT` | `NOT NULL DEFAULT (datetime('now','localtime'))`. |
| *table constraint* | | `UNIQUE (user_id, event_id)`: one registration per student per event. |

## 4.3 Conventions

**Times as text.** Times are `TEXT` in `'YYYY-MM-DD HH:MM'` format. Because the most significant part comes first, string comparison equals time comparison, so `<`, `>`, `BETWEEN` and `ORDER BY` work directly. `now_string()` in `events.py` produces the current time in the same format. HTML `datetime-local` inputs use a `T` separator, converted by `to_db_time()`.

**Foreign keys.** SQLite enforces them only when `PRAGMA foreign_keys = ON` is run on the connection; `get_db()` does this every time.

**Rows as dicts.** `row_factory = sqlite3.Row`, so code writes `row["title"]`. Report functions convert rows with `dict(row)` before adding computed keys.

**Seats left** are never stored; they are computed as `capacity - COUNT(r.id)` with a `LEFT JOIN` on `registrations`, so they can never get out of step.

### 4.4 Indexes

`sql/indexes.sql` is deliberately separate from the schema: the indexes were the result of profiling, and keeping them apart lets the profiler measure with and without them (`APPLY_INDEXES=0`).

| Index | Columns | Speeds up |
|---|---|---|
| `idx_registrations_event` | `registrations(event_id)` | every seats-left count and report `JOIN`; covering index for counts |
| `idx_events_venue_start` | `events(venue_id, start_time)` | venue filter in search, venue clash check |
| `idx_events_category` | `events(category)` | category filter in search |
| `sqlite_autoindex_registrations_1` | `(user_id, event_id)` | created automatically by the `UNIQUE` constraint; serves "is this student registered?" and My registrations |

Table 2: Indexes. An extra index on `registrations(user_id)` was rejected as redundant.

## 5 Route reference

| URL | Method | Access | Endpoint and purpose |
|---|---|---|---|
| `/` | GET | all | `index`: home page with counts |
| `/auth/register` | GET/POST | all | `auth.register`: student sign-up |
| `/auth/login` | GET/POST | all | `auth.login` |
| `/auth/logout` | GET | login | `auth.logout` |
| `/events/` | GET | all | `events.list_events`: upcoming events; query params `q`, `category`, `venue_id`, `date_from`, `date_to` |
| `/events/<id>` | GET | all | `events.event_detail` |
| `/events/<id>/register` | POST | login | `events.register_for_event` |
| `/events/<id>/cancel` | POST | login | `events.cancel_registration`; form field `return_to=my` returns to My registrations |
| `/my-registrations` | GET | login | `events.my_registrations` |
| `/admin/` | GET | admin | `admin.dashboard` |
| `/admin/venues` | GET | admin | `admin.list_venues` |
| `/admin/venues/new` | GET/POST | admin | `admin.create_venue` |
| `/admin/venues/<id>/edit` | GET/POST | admin | `admin.edit_venue` |
| `/admin/venues/<id>/delete` | POST | admin | `admin.delete_venue` |
| `/admin/events` | GET | admin | `admin.list_events`: all events incl. past |
| `/admin/events/new` | GET/POST | admin | `admin.create_event` |
| `/admin/events/<id>/edit` | GET/POST | admin | `admin.edit_event` |
| `/admin/events/<id>/delete` | POST | admin | `admin.delete_event` |
| `/admin/events/<id>/participants` | GET | admin | `admin.participants` |
| `/admin/events/<id>/participants.csv` | GET | admin | `admin.participants_csv`: CSV download |
| `/admin/reports` | GET | admin | `admin.reports_page` |

| URL | Method | Access | Endpoint and purpose |
|---|---|---|---|
| /static/<path> | GET | all | static: style.css |

Table 3: All 22 URL rules (from `flask -app app routes`). *login*: visitors are redirected to the login page. *admin*: visitors are redirected, students get 403. `<id>` is an integer; anything else gives 404.

# 6 Module reference

Signatures are shown in grey boxes. "Returns" describes the value; routes return a rendered page or a redirect.

## 6.1 app/db.py: database access

```
get_db() -> sqlite3.Connection
```

Returns the connection for the current request, opening it on first use and storing it in `g.db`. Sets `row_factory = sqlite3.Row` and `PRAGMA foreign_keys = ON`.

```
close_db(error=None)
```

Closes `g.db` if one was opened. Registered with `app.teardown_appcontext`, so it runs after every request and app context.

```
init_db()
```

Runs `sql/schema.sql` with `executescript`. Destroys existing data.

```
add_indexes()
```

Runs `sql/indexes.sql`. Idempotent (`IF NOT EXISTS`).

```
init_db_command / add_indexes_command / create_admin_command
```

Click commands `init-db`, `add-indexes`, `create-admin -name -email -password`. `create-admin` skips an existing email and stores `generate_password_hash(password)`.

```
init_app(app)
```

Registers `close_db` and the three commands on the app.

## 6.2 app/auth.py: accounts and access control

```
class User(UserMixin)
```

Built from a `users` row; attributes `id`, `name`, `email`, `role`; property `is_admin`. `UserMixin` supplies `is_authenticated`, `get_id()` and friends.

```
load_user(user_id) -> User | None
```

The Flask-Login user loader: one `SELECT` by id on every request that uses `current_user`. Returns `None` for a deleted user, which logs them out.

```
admin_required(view)
```

Decorator. Not logged in: `login_manager.unauthorized()` (redirect to login). Logged in but not admin: `abort(403)`. Uses `functools.wraps` so Flask still sees the original function name as the endpoint.

`register()`

GET shows the form; POST validates name (non-empty), email (contains `@`, lower-cased) and password (at least 6 characters), then inserts with role `'student'`. A duplicate email raises `sqlite3.IntegrityError`, shown as a friendly message. Logged-in users are redirected.

`login()`

Looks up the email, verifies with `check_password_hash`, calls `login_user(User(row))`. The same message is used for an unknown email and a wrong password.

`logout()`

`@login_required`; calls `logout_user()` and redirects home.

### 6.3 app/events.py: browsing and registrations

`CATEGORIES = ["Technical", "Cultural", "Sports", "Workshop", "Seminar"]`

The single list of categories, imported by `admin.py` and `seed.py`.

`now_string() -> str`

Current local time as `'YYYY-MM-DD HH:MM'`.

`get_event(event_id) -> Row | None`

One event joined with its venue and organiser, plus `seats_left`. `LEFT JOIN registrations ... GROUP BY e.id`.

`is_registered(user_id, event_id) -> bool`

`SELECT 1` on the unique pair.

`build_search_sql(keyword="", category="", venue_id=None, date_from="", date_to="") -> (str, list)`

Builds the search query. Starts with `e.start_time >= now`; adds a condition for each non-empty filter. `conditions` only ever contains fixed SQL written in the code; user input goes into `params`. Keyword matches title or description with `LIKE '%text%'` (case-insensitive for ASCII). `date_to` has `" 23:59"` appended so the whole day is included. Separate from execution so `profiling/explain.py` can explain exactly the same SQL.

`search_events(...) -> list[Row]`

Runs `build_search_sql` with the same arguments.

`list_events()`

Reads filters from `request.args` (`venue_id` with `type=int`, so bad input becomes `None`), calls `search_events`, loads venues for the dropdown and passes the filters back to keep the form filled in.

`event_detail(event_id)`

404 if missing; computes `is_registered` and `has_started` for the template's action area.

`register_for_event(event_id)`

POST, login required. Applies the registration rules (Section 7) and inserts; `registered_at` comes from the column default.

`cancel_registration(event_id)`

POST, login required. Refuses after the start; otherwise `DELETE` and uses `cursor.rowcount` to tell whether a row existed. Redirects to My registrations when `return_to=my`, otherwise to the detail page.

`my_registrations()`

The current user's upcoming registrations joined with event and venue, ordered by start time.

### 6.4 app/admin.py: administration

All routes are decorated with `@admin_required`.

`to_db_time(value) -> str | None`

'2026-11-20T14:30' → '2026-11-20 14:30'; `None` if it does not parse.

`find_clash(venue_id, start_time, end_time, ignore_event_id=None) -> Row | None`

First event at the venue whose interval overlaps: `start_time < new_end AND end_time > new_start`. Touching intervals do not clash. `ignore_event_id` excludes the event being edited.

`read_event_form(form, event_id=None) -> (dict, None) | (None, str)`

Validates and normalises the event form (Section 7). Returns the clean values or the first error message.

`read_venue_form(form, venue_id=None) -> (dict, None) | (None, str)`

Name and location required, capacity a positive integer; when editing, not below `MAX(capacity)` of the venue's events.

`get_participants(event_id) -> list[Row]`

Name, email, `registered_at`, earliest first.

`dashboard()`

One query with five scalar sub-queries: events, upcoming, venues, students, registrations.

`list_venues(), create_venue(), edit_venue(venue_id), delete_venue(venue_id)`

Venue CRUD. Forms redisplay the submitted values on error. Delete refuses while events use the venue (the foreign key would also block it).

`list_events(), create_event(), edit_event(event_id), delete_event(event_id)`

Event CRUD. `created_by` is `current_user.id`. Edit pre-fills `datetime-local` values by replacing the space with `T`. Delete relies on `ON DELETE CASCADE` to remove registrations and returns 404 when nothing was deleted.

`all_venues(), get_event_or_404(event_id)`

Small helpers for the event form dropdown and the participant pages.

`participants(event_id), participants_csv(event_id)`

HTML list, and the same rows written with `csv.writer` into `io.StringIO`, returned as `text/csv` with `Content-Disposition: attachment; filename=event_<id>_participants.csv`.

`reports_page()`

Renders `admin/reports.html` with the three report functions.

### 6.5 app/reports.py: report queries

The SQL is held in module constants so `profiling/explain.py` can explain the same text.

`percent(part, whole) -> float`

`round(100 * part / whole, 1)`, or 0.0 when `whole` is 0. Done in Python because Python and SQLite round halves differently, and the numbers had to match the first version exactly.

`event_stats() -> list[dict]`

`EVENT_STATS_SQL`: every event with `registered` (`LEFT JOIN` + `COUNT(r.id)`), sorted by registrations, start, id; adds `fill_rate`.

`popular_categories() -> list[dict]`

`CATEGORIES_SQL`: per category, `COUNT(DISTINCT e.id)` events (the join repeats events) and `COUNT(r.id)` registrations.

`venue_utilisation() -> list[dict]`

`VENUES_SQL`: a derived table counts registrations per event first, then the outer query sums events, registrations and capacity per venue (so capacity is not multiplied by the number of registrations). `LEFT JOIN` keeps venues without events. Adds `utilisation`.

```sql
SELECT v.id, v.name, v.location,
       COUNT(e.id) AS events,
       COALESCE(SUM(e.registered), 0) AS registrations,
       COALESCE(SUM(e.capacity), 0)   AS seats_offered
FROM venues v
LEFT JOIN (
    SELECT ev.id, ev.venue_id, ev.capacity, COUNT(r.id) AS registered
    FROM events ev
    LEFT JOIN registrations r ON r.event_id = ev.id
    GROUP BY ev.id
) AS e ON e.venue_id = v.id
GROUP BY v.id
ORDER BY registrations DESC, v.name
```

Listing 2: VENUES_SQL: the most involved report query.

### 6.6 app/seed.py: demo data

`seed_command(num_events=50, num_students=100)`

The `flask seed` command. Seeds `random` and Faker with 42 for reproducible data. Requires an admin (the organiser). Deletes registrations, events, venues and students, resets their `sqlite_sequence` counters, inserts the 10 fixed `VENUES`, students with one shared pre-computed password hash (hashing is deliberately slow), events on random non-clashing slots in the next 60 days (gives up after `20 * N` attempts), and for each event a `random.sample` of distinct students no larger than its capacity.

`overlaps(start, end, booked_slots) -> bool`

The same overlap rule as `find_clash`, on Python `datetime` pairs.

## 7 Business rules

### 7.1 Registration

`register_for_event` checks, in this order, and stops at the first failure:

1. user is not an admin;
2. `start_time > now` (the event has not started);
3. no existing registration (also guaranteed by `UNIQUE (user_id, event_id)`);
4. `seats_left > 0`.

Cancellation is allowed only before the event starts.

### 7.2 Event validation (`read_event_form`)

1. title not empty;
2. category in `CATEGORIES`;
3. start and end are valid `datetime-local` values;
4. end after start (also a `CHECK` constraint);
5. capacity is an integer above 0 (also a `CHECK`);
6. venue exists, and capacity $\leq$ venue capacity;
7. when editing: capacity $\geq$ current registrations;
8. no clash at the venue (`find_clash`, ignoring the event itself when editing).

### 7.3 The overlap rule

Two half-open intervals $[s_1, e_1)$ and $[s_2, e_2)$ overlap exactly when $s_1 < e_2 \wedge e_1 > s_2$. This single condition covers partial overlap on either side, containment in both directions and identical intervals, while letting back-to-back events share a boundary.

### 7.4 Venue rules

Name and location required; capacity a positive integer; on edit, not smaller than the largest event at the venue; delete only when no events use the venue.

## 8 Templates and styling

### 8.1 Template inheritance

Every page extends `templates/base.html`, which defines the `title` and `content` blocks, the navigation bar (links depend on `current_user`), the flash-message area and the footer. Jinja2 auto-escapes all variables, so user-supplied text such as event descriptions cannot inject HTML.

| Template | Rendered by / purpose |
|---|---|
| `base.html` | layout shared by all pages |
| `index.html` | `index`: welcome text and counts |
| `error.html` | 403 and 404 handlers (`code`, `message`) |
| `auth/login.html`, `auth/register.html` | login and sign-up forms |
| `events/list.html` | search form and event cards |
| `events/detail.html` | one event and the Register / Cancel action area |
| `events/my_registrations.html` | student's upcoming registrations with Cancel buttons |
| `admin/dashboard.html` | counters and shortcuts |
| `admin/venues.html`, `admin/venue_form.html` | venue list; shared new/edit form (`venue_id` is `None` for new) |
| `admin/events.html`, `admin/event_form.html` | event list; shared new/edit form |
| `admin/participants.html` | participant table and Export CSV button |
| `admin/reports.html` | three report tables and CSS fill-rate bars |

Table 4: The 15 templates.

**Jinja comments**

Jinja processes tags even inside HTML comments, so `<!-- {% block %} -->` breaks the template. Use Jinja's own comment syntax `{# ... #}` for comments that mention template tags.

### 8.2 CSS

`static/style.css` is the only stylesheet. Colours and sizes are CSS custom properties on `:root` (`--primary: #1d4ed8`, `-error`, `-radius` ...), so the palette is changed in one place. Layout uses flexbox (header, sticky footer, search form that wraps) and CSS grid (`repeat(auto-fill, minmax(280px, 1fr))` for event cards, so the number of columns follows the screen width). Wide tables sit in a wrapper with `overflow-x: auto` for phones. Fill-rate bars on the reports page are a `span` inside a `div` whose `width` style is the percentage.

## 9 Security model

| Threat | Mitigation |
|---|---|
| SQL injection | every value through `?` placeholders; SQL text never built from input |
| HTML injection (XSS) | Jinja2 auto-escaping |
| Stolen passwords | salted, slow hashes (Werkzeug); generic login error message |
| Privilege escalation | role fixed to `'student'` at sign-up; `admin_required` on every admin route; DB `CHECK` on role |
| Cross-site requests | all state changes are POST; `SameSite=Lax` session cookie |
| Accidental GET side effects | delete/register/cancel are forms, never links |
| Bad IDs | `<int:...>` URL converters and 404s |

Table 5: Security measures.

**Known limitations.** No CSRF tokens (`SameSite` covers most cases); the seat check and the insert are two statements, so two simultaneous requests for the last seat could over-book by one; `/auth/logout` is a GET; the default `SECRET_KEY` must be replaced in production; keyword search cannot use an index because of the leading `%` wildcard.

## 10 Testing

Tests live in `tests/` and run with `./scripts/test.sh`. `pytest.ini` sets `testpaths = tests` and `pythonpath = .` so tests can `import app`.

| Fixture | Provides |
|---|---|
| `app` | `create_app` with `TESTING=True` and a database in `tmp_path`; schema applied |
| `client` | `app.test_client()`: a fake browser |
| `auth` | `AuthActions` with `register()` and `login()` helpers |
| `admin_user` | an admin inserted directly with SQL |
| `sample_events` | one venue and four events (three upcoming, one past); returns `{title: id}` |
| `student_client` | client logged in as a newly registered student |
| `admin_client` | client logged in as the admin |

Table 6: Fixtures in `tests/conftest.py`.

When you add a feature, add tests next to the related ones and run the whole suite. The Test Plan & Report lists every existing test case.

## 11 Profiling toolkit

**`scripts/profile.sh <label> [N]`** creates `profiling/profile.db` (via `CAMPUS_DB`), seeds N events, optionally adds indexes, then runs the two modules below.

**profiling/run_profile.py** for each workload in WORKLOADS: RUNS timed runs with time.perf_counter() and SQL statements counted with conn.set_trace_callback; then one run under cProfile. Writes profile.prof, profile_stats.txt (top 20 by cumulative time) and timings.txt.

**profiling/explain.py** writes EXPLAIN QUERY PLAN output for the search variants, the naive per-event count, the three report queries, the clash check and the "is registered" query to query_plans.txt.

To profile a new function, add a ("name", function) pair to WORKLOADS, and its SQL to queries() in explain.py. View a profile with snakeviz profiling/<label>/profile.prof. The saved results in profiling/before/ are the evidence of the naive version and must not be overwritten; results and analysis are in profiling/RESULTS.md and the project report.

# 12 Extending the application

## 12.1 Add a page

1. Add a view function to the right Blueprint, with @login_required or @admin_required as needed.
2. Put any logic worth testing in a plain function next to it.
3. Create a template whose first line is {% extends "base.html" %}, and link to the page with url_for("blueprint.function").
4. Add tests for the happy path, each refusal, and access control.

## 12.2 Change the schema

1. Edit sql/schema.sql (constraints belong in the database where possible).
2. Update every INSERT/SELECT that touches the table, the seed command and fixtures.
3. Recreate the database with ./scripts/setup.sh (there are no migrations; this wipes data).
4. If a new query filters or joins on a column, check its plan with EXPLAIN QUERY PLAN and add an index to sql/indexes.sql only if it helps.

## 12.3 Add a category or a report

Categories: add the name to CATEGORIES in events.py (and optionally to TITLES in seed.py). Reports: write one SQL query as a constant in reports.py, a function that runs it, a table in admin/reports.html, and a test with exact expected numbers.

## 12.4 Coding conventions

- PEP 8 style, descriptive names, a docstring on every function, comments that explain why.
- SQL keywords in upper case; one clause per line in longer queries; table aliases e, v, r, u.
- Every data-changing request is POST and ends in a redirect.
- Bash scripts: set -euo pipefail, source common.sh, clear echo progress messages, LF line endings (enforced by .gitattributes).

### 12.5 Git workflow

Small commits on `main`, each leaving the tests passing, using Conventional Commit prefixes: `feat:`, `fix:`, `test:`, `perf:`, `chore:`, `docs:`. Generated files (`venv/`, `*.db`, `__pycache__`, LaTeX build files) are in `.gitignore`.